

实验 3：软件架构与界面设计

个人项目：面向数模的特征衍生系统 (FeaturePilot)

软件工程一班 24111302025 罗心月

2026 年 9 月 15 日

1 实验目的

- 将实验 2 的需求落实为可开发的模块、数据、接口和界面。
- 完成关键架构决策与风险替代方案。
- 形成实验 4 可直接使用的设计基线。

2 需求输入与设计范围

2.1 核心流程

上传 CSV 与赛题背景 → 本地体检 → 语义推荐 → 本地校验 → 人工确认 → 导出 Pandas 代码。

用户故事：作为数模参赛者，我希望快速得到少量、可解释且可运行的衍生特征，以便缩短特征工程时间。

编号	核心需求	设计落实	验收条件
FR-01	上传 CSV 并设置背景、Target	上传页、app.py	合法 CSV 进入体检；错误文件给出原因
FR-02	本地画像与风险扫描	profiler.py、字段画像	输出缺失率、偏度、相关性与风险级别
FR-03	推荐可解释特征	recommender.py、推荐卡片	返回 3-10 条 {name, op, params, reason}
FR-04	安全执行与代码生成	codegen.py、算子注册表	未注册算子或未知列被拒绝，不使用 exec/eval
FR-05	高风险人工确认	风险确认页、确认记录	红/黄项默认锁定，确认后才可导出
NFR-01	隐私	原始数据仅本地处理	发往模型的内容不含原始行数据
NFR-02	性能	超大数据先抽样 2 万行	常规数据体检目标 ≤ 3 秒
NFR-03	可靠性	本地规则降级	模型超时时时仍可完成基础推荐

2.2 用例设计



Figure 1: 图 1 FeaturePilot 用例图

图 1 说明：参赛者参与上传、体检、推荐、预览、确认和导出；LLM 只参与语义推荐。风险确认和本地降级属于条件触发的扩展用例。

2.3 普通程序与 AI 边界

责任方	主要职责	检查方式
用户	提供背景、选择 Target、确认风险	页面确认与最终选择
普通程序	统计、风险扫描、算子执行、代码模板	类型/列名/数值边界校验
LLM	理解字段语义，给出机理解释与受控算子意向	JSON Schema + 本地白名单
工具库	Pandas/NumPy 计算与可视化	单元测试、预览结果

3 技术选型与架构决策

3.1 候选方案

方案	熟悉度	成本	AI 接入	测试/部署	结论
Python + Streamlit	高	低	易	单机启动简单	采用
Python + FastAPI + Vue	中	中	易	前后端工作量较大	暂不采用
Java + Spring Boot	中	高	中	对数据分析生态不友好	不采用

3.2 简要 ADR

决策	选择	理由	不足/替代
ADR-01 界面框架	Streamlit 单体应用	一人可快速完成，适合数据看板	复杂交互受限；后续可换 FastAPI+Vue
ADR-02 数据保存	会话内存 + 本地导出文件	不保存原始 CSV，隐私和实现成本更优	刷新后状态丢失；后续可加 SQLite
ADR-03 AI 输出	JSON Schema + 受控算子	可校验、可审计、避免任意代码	算子范围有限；按需扩展注册表
ADR-04 架构风格	轻量分层/模块化	UI、业务、外部依赖边界清晰	单机扩展性一般，但满足 MVP

关键依赖替代：云端模型不可用时，recommender.py 切换本地规则引擎；Streamlit 受限时保留业务模块，仅替换界面层。

4 软件架构与模块设计



Figure 2: 图 2 软件与外部环境关系图

图 2 说明：原始 CSV 只进入本地程序；云端模型只接收统计摘要、背景和 risk 名单。



Figure 3: 图 3 总体架构与模块结构图

图 3 说明：界面层调用业务层；codegen.py 只通过白名单算子执行推荐。

模块	职责	直接依赖
app.py	会话、上传、导航、结果反馈	三个 core 模块
profiler.py	数据画像、打分、风险扫描	Pandas、scikit-learn
recommender.py	LLM 推荐、本地降级	模型 API、Profile
codegen.py	参数校验、预览执行、脚本拼装	NumPy、算子注册表

5 数据、接口与核心流程设计

5.1 数据模型

图 4 说明：MVP 不持久化原始 CSV；confirmed_at 用于追踪高风险特征的人工确认。

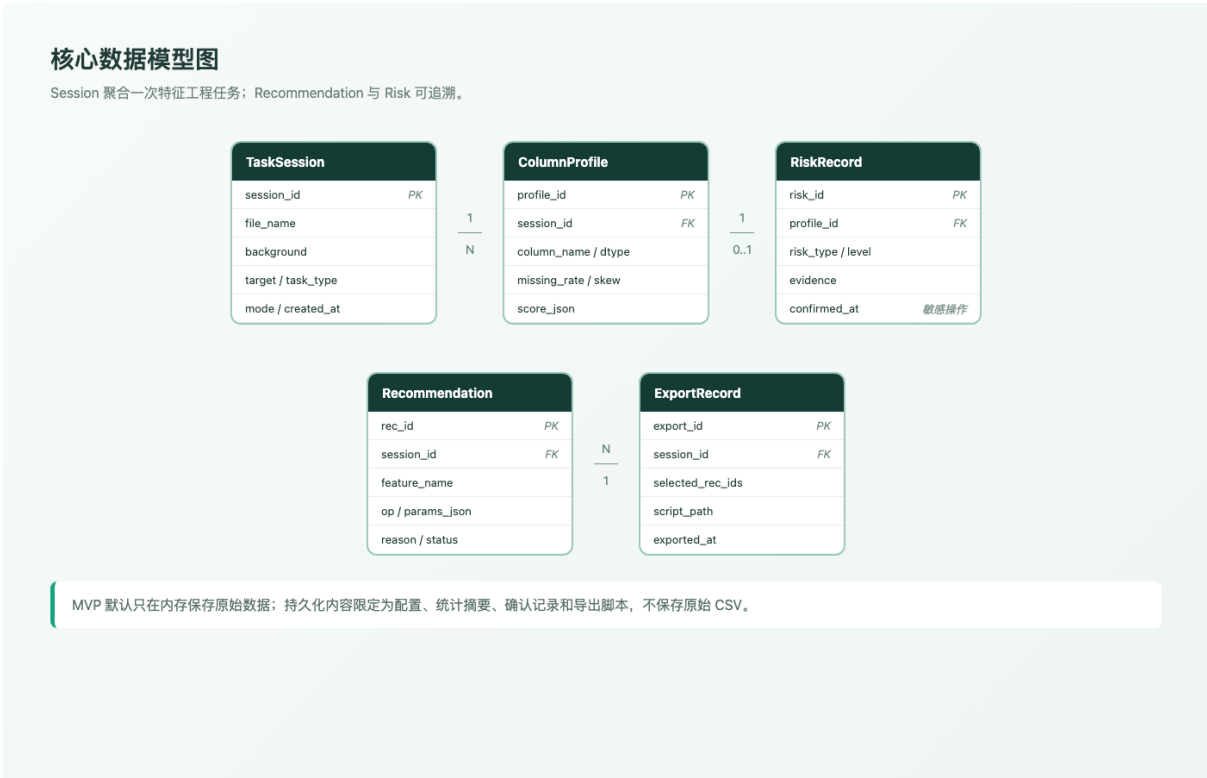


Figure 4: 图 4 核心数据模型图

5.2 核心类图

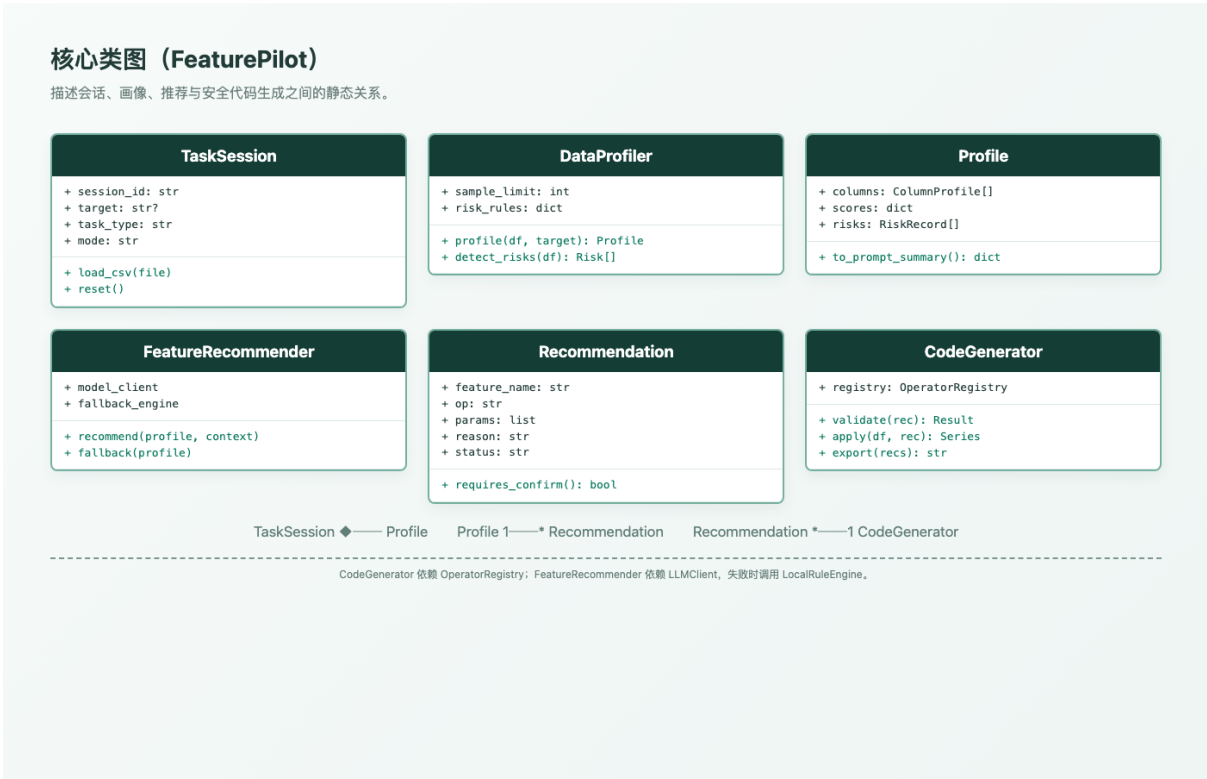


Figure 5: 图 5 FeaturePilot 核心类图

图 5 说明：TaskSession 聚合画像；FeatureRecommender 生成推荐；CodeGenerator 通过算子注册表完成校验、预览和导出。

5.3 主要接口

接口	提供/调用	输入	输出	失败
profile(df,target,bask)	ker / app	DataFrame、目标列、任务类型	Profile	空表、列不存在、类型错误
recommend(profile,context)	render / app	摘要、背景、风险名单	Recommendation[]	超时后切本地规则
validate_and_apply(df,rec)	app	数据副本、推荐 JSON	新特征、校验记录	未知算子/列直接拒绝
export_code(recs)	codegen / app	已选择且已解锁推荐	.py 文本	空选择提示用户

正常调用：

```
{"feature_name":"pressure_volume_work","op":"interaction","params":["pressure","volume"],"reason":""}
```

错误调用：

```
{"feature_name":"x","op":"python_eval","params":["missing_col"]}
```

处理结果：op 未注册且列名不存在，整条建议拒绝并写入校验日志。

5.4 核心流程

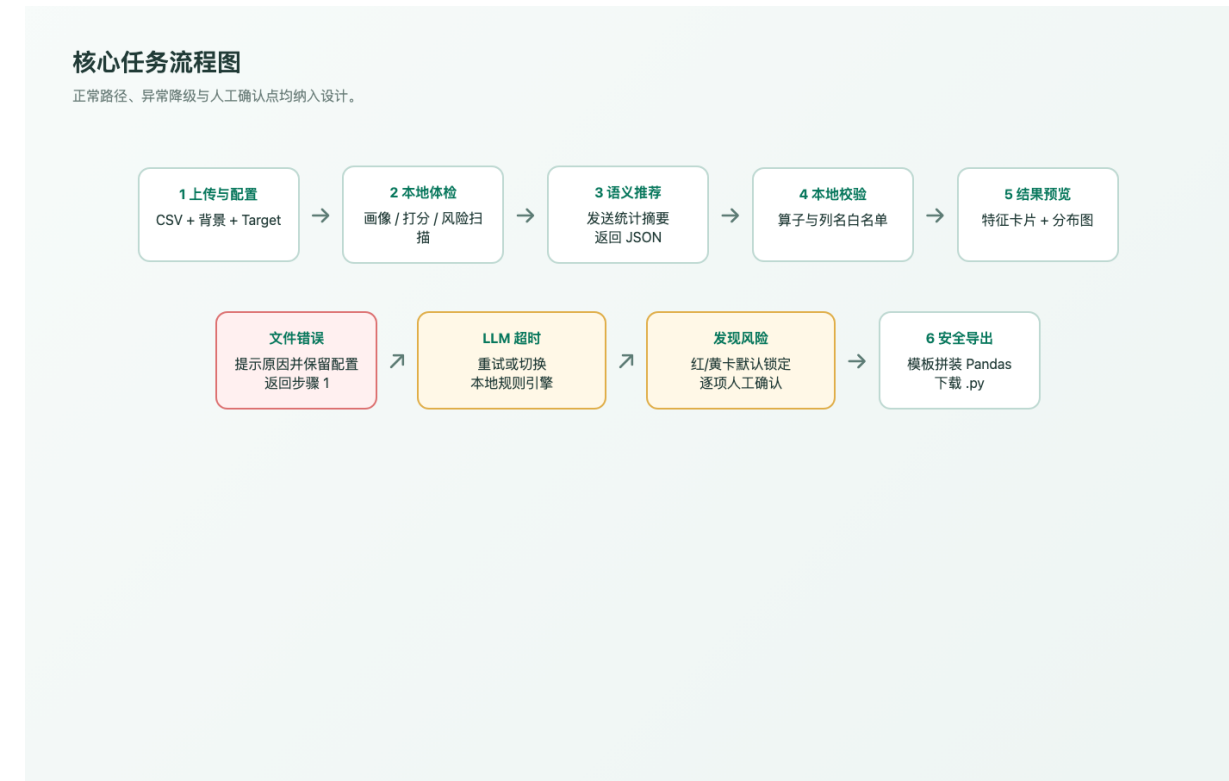


Figure 6: 图 6 核心任务流程图

图 6 说明：文件错误回到上传页；模型超时可降级；风险项必须人工确认。

6 界面原型与交互设计

图 7 说明：主路径线性推进，允许返回、重试和重新推荐。

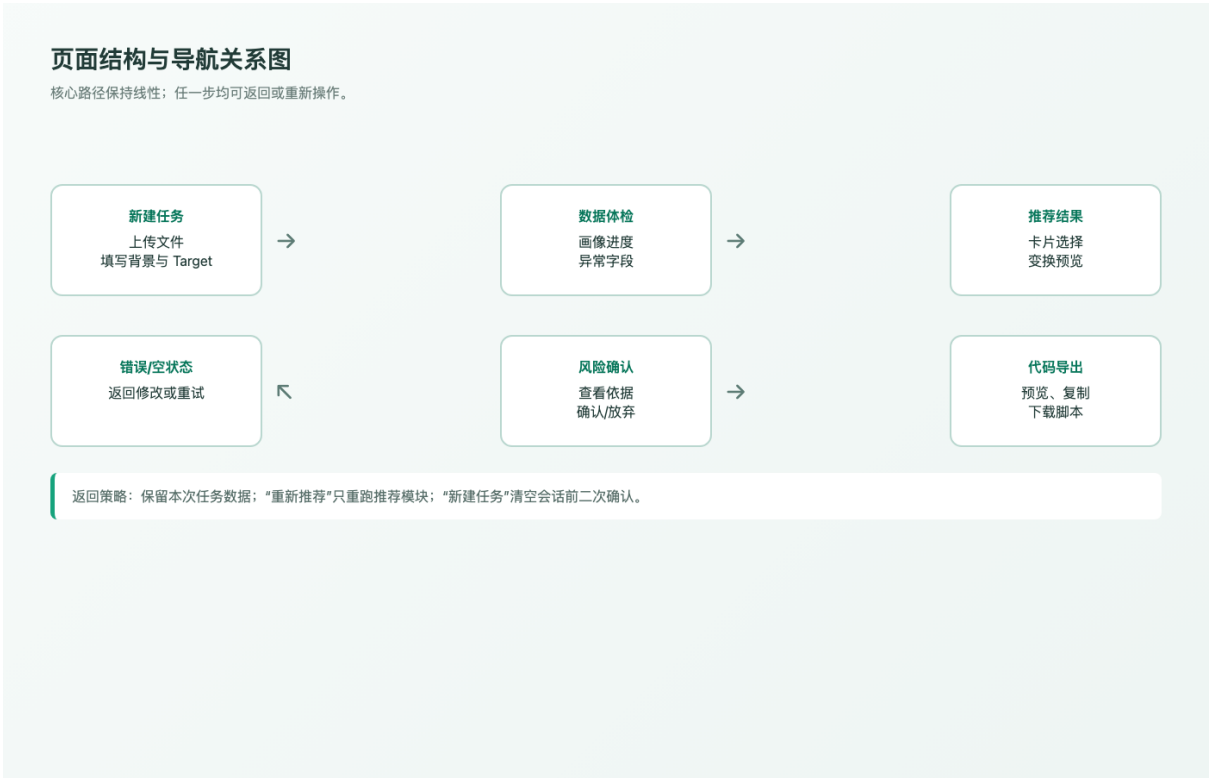


Figure 7: 图 7 页面结构与导航关系图

图 8 操作记录：选择 CSV，填写赛题背景、Target 和任务类型。页面明确提示原始数据不上传。

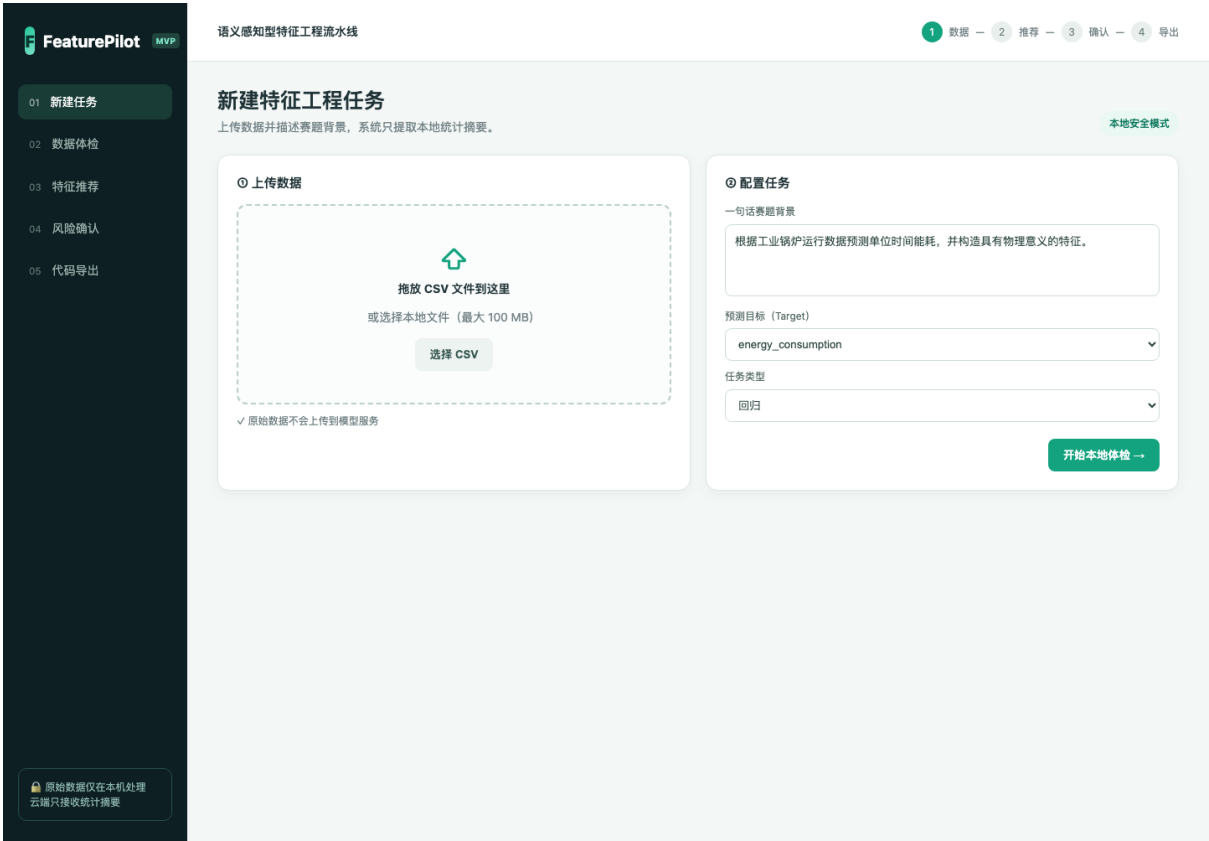


Figure 8: 图 8 首页/任务配置页

图 9 操作记录：显示字段数、风险数、分析耗时和字段级风险，加载时给出当前阶段。

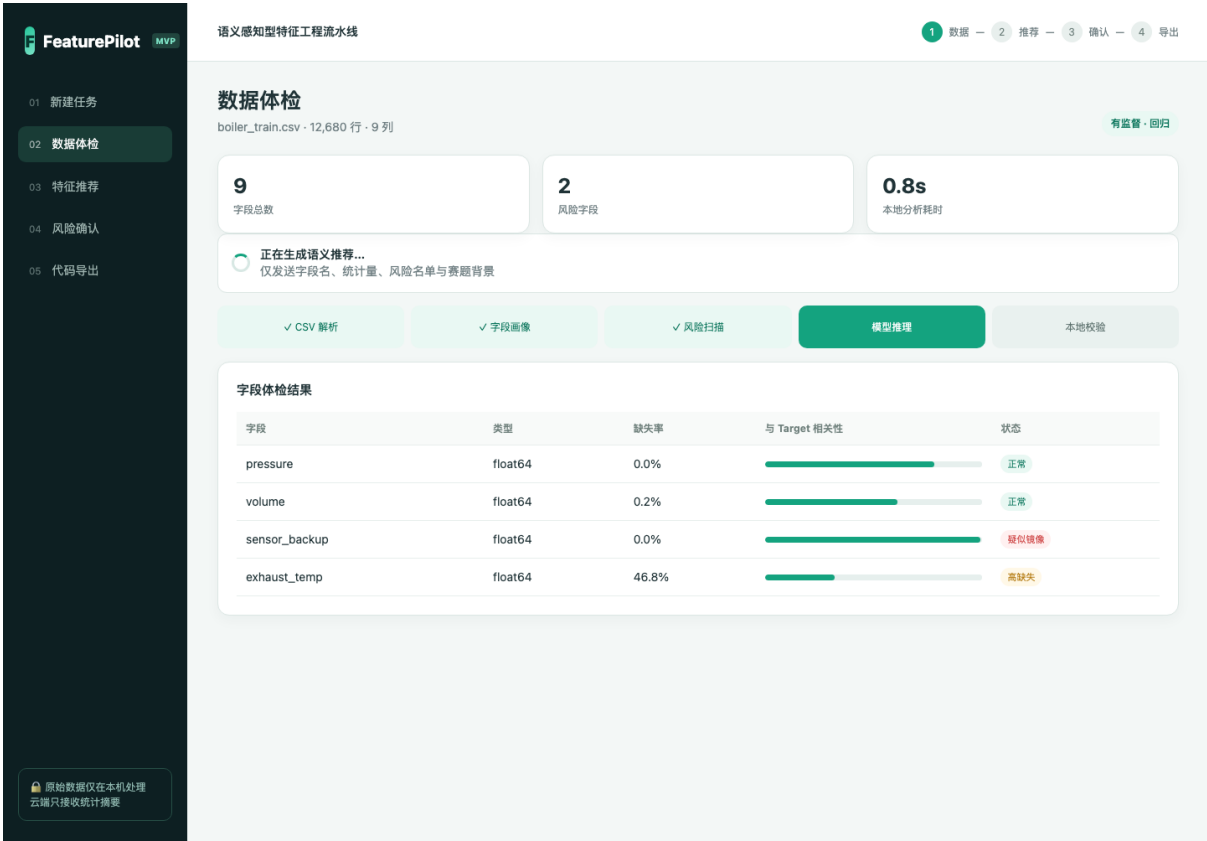


Figure 9: 图 9 数据体检与加载状态

图 10 操作记录：安全项可直接选择；临界项锁定；变换前后分布并排显示。

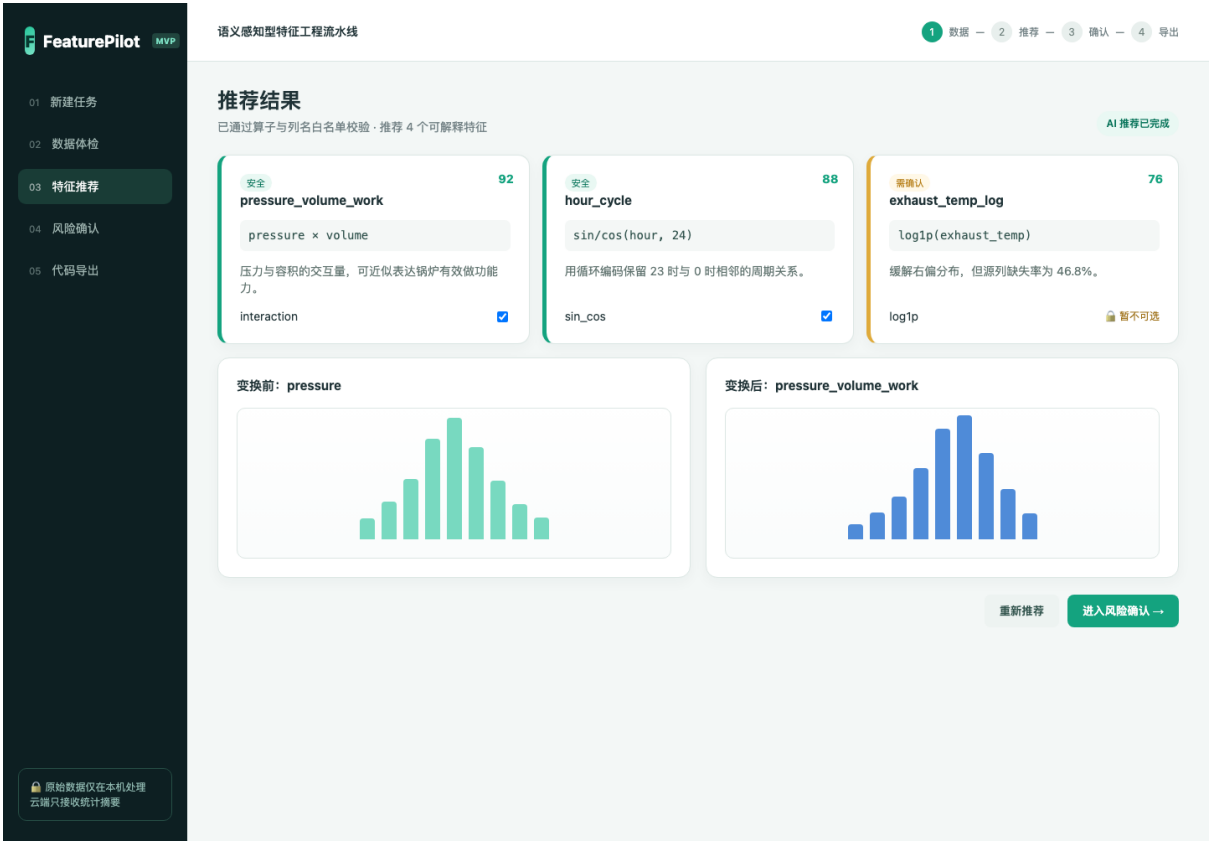


Figure 10: 图 10 特征推荐与分布预览页

图 11 操作记录：黄卡可逐项确认，镜像/泄露红卡默认阻断，避免误导出。

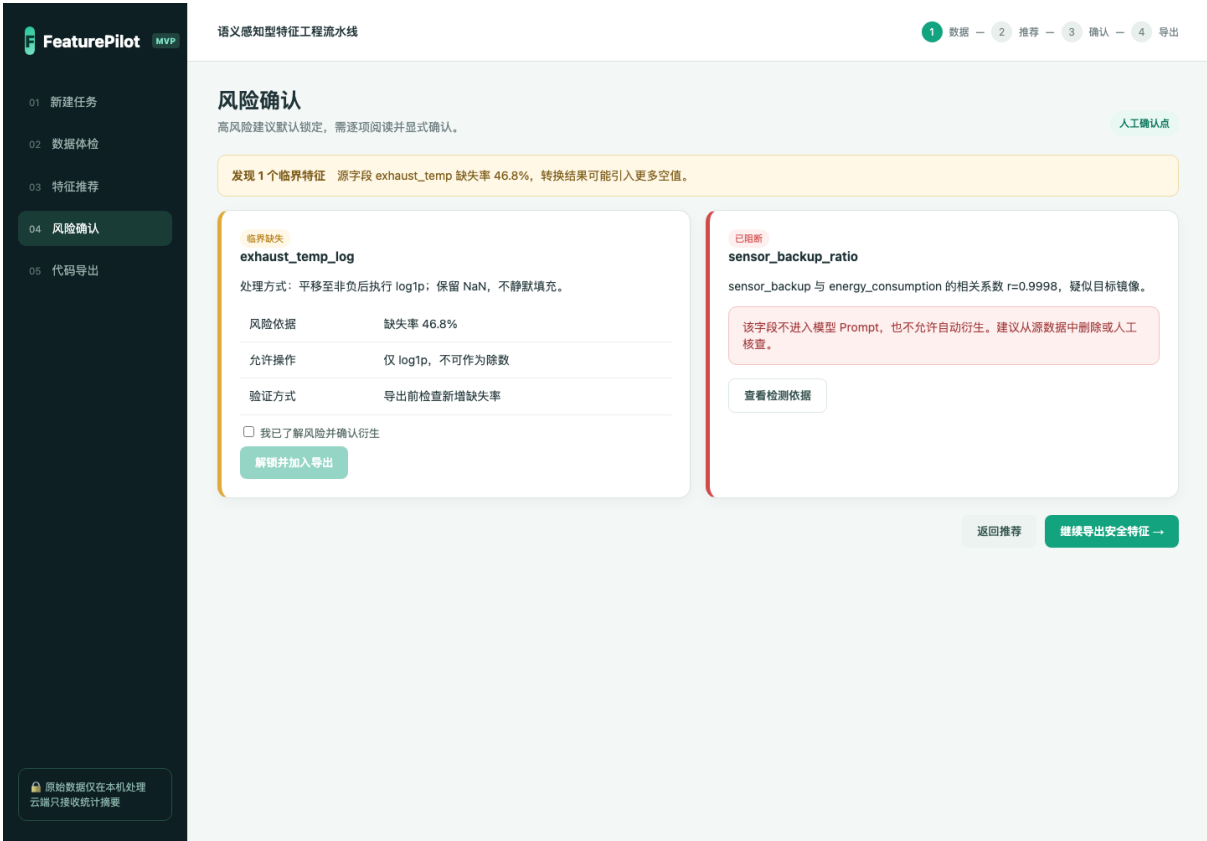


Figure 11: 图 11 风险确认页

图 12 操作记录：左侧核对清单与状态，右侧预览模板生成的 Pandas 代码。

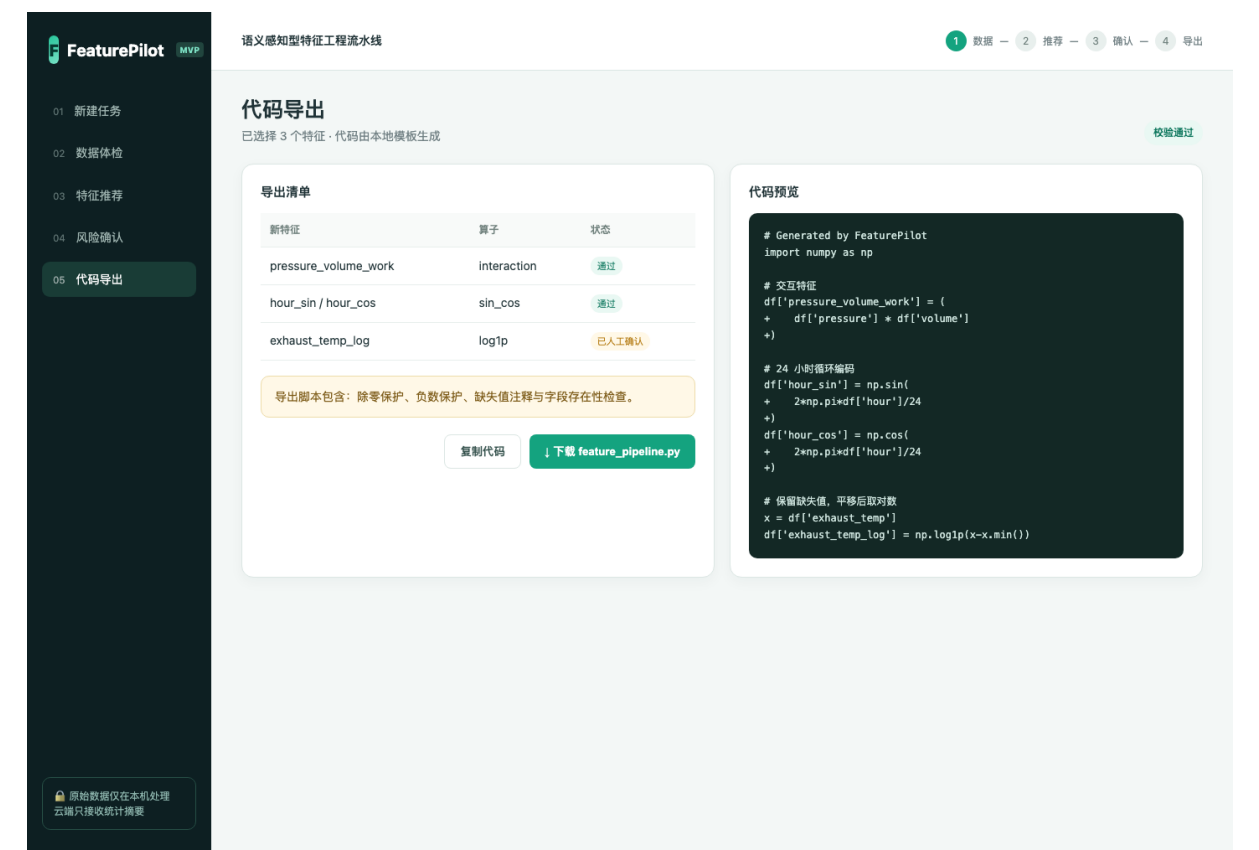


Figure 12: 图 12 代码导出页

图 13 操作记录：保留本地体检结果，可重连模型或切换本地规则，不让流程中断。

6.1 元素与反馈

类型	设计
静态元素	阶段进度、隐私提示、风险规则、公式说明
输入元素	CSV、背景文本、Target、任务类型、复选框
命令元素	开始体检、重新推荐、确认解锁、复制/下载
动态元素	进度、指标、推荐卡片、分布图、代码预览
状态反馈	加载、空数据、格式错误、模型超时、风险阻断、导出成功

7 设计一致性检查与版本保存

需求	架构模块	数据/接口	界面	检查结果
FR-01	app、profiler	TaskSession、profile	图 8、图 9	已落实
FR-02	profiler	ColumnProfile、RiskRecord	图 4、图 9	已落实
FR-03	recommender	Recommendation、recommend	图 1、图 5、图 10	已落实
FR-04	codegen	validate/apply/export	图 5、图 12	已落实
FR-05	app、codegen	RiskRecord.confirmed	图 4、图 11	已落实
NFR-01~03	全模块	摘要传输、抽样、降级	图 2、图 6、图 13	已落实

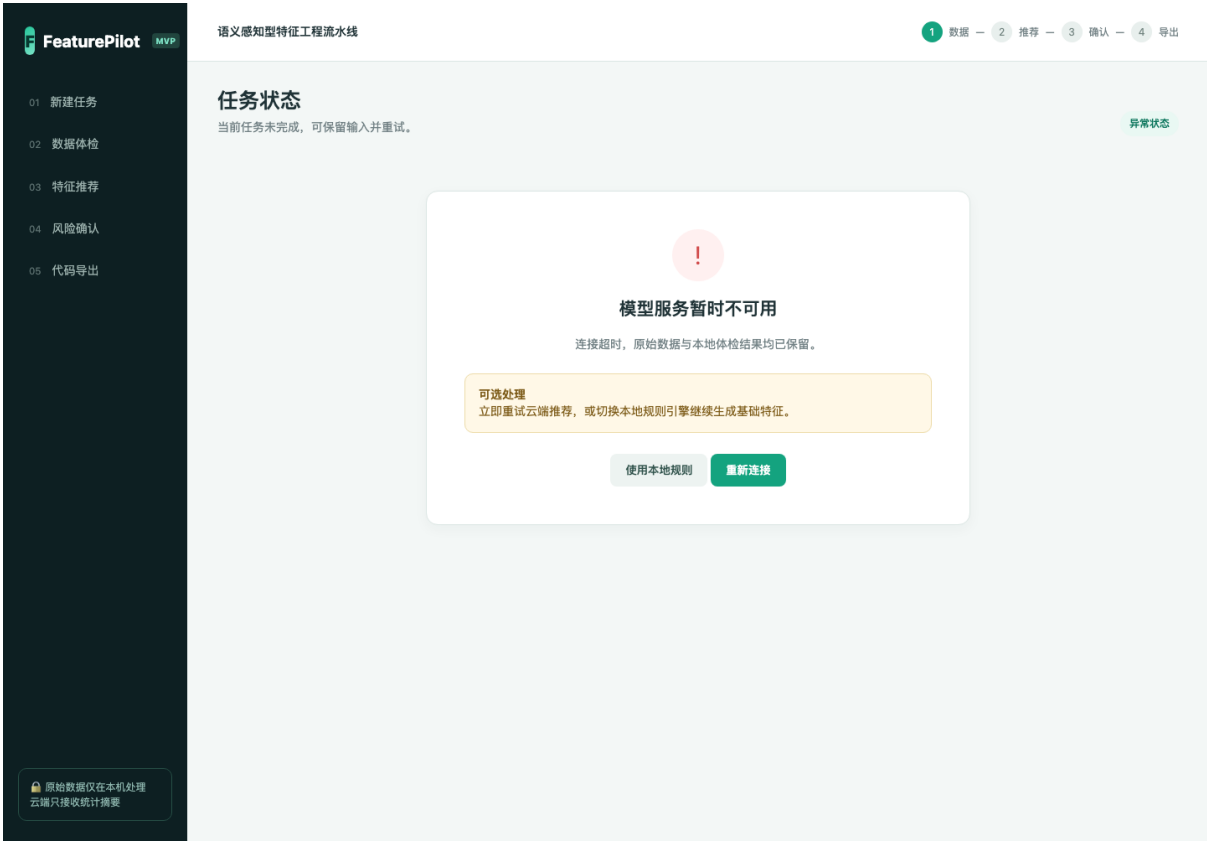


Figure 13: 图 13 模型服务异常与重试页

检查发现并调整：

- 1. 初稿只有推荐成功页，补充加载、模型超时与本地降级状态。
- 2. 初稿风险特征可直接勾选，改为默认锁定并逐项确认。
- 3. 初稿允许模型返回公式字符串，改为 `op + params` 受控结构。

版本证据：设计图、原型与截图保存在 `prototype/`、`assets/experiment3/`；本地 Git 提交为 **6dfe01d** (原型与截图)、**cc019a6** (补充用例图与核心类图)。

8 实验小结

本实验完成了用例图、核心类图、需求追踪、技术选型、分层架构、数据与接口、异常流程及 6 种界面状态。最终设计坚持“本地算事实、模型做语义、程序强校验、风险交给人确认”，可直接作为实验 4 的开发基线。